Worksheet — 1.4 Data Types, Data Structures & Boolean Algebra

Name: ____ Date: __

OCR A Level Computer Science (H446) · Component 01 · 1.4.1 Data types · 1.4.2 Data structures · 1.4.3 Boolean algebra

Time: ~60 min. SHOW ALL WORKING.

Section A — Skills practice

Activity 1 — Conversion drill (denary ↔ binary ↔ hex)

All binary values are **8-bit unsigned**. Fill in every blank cell. Use the place-value headers to help.

`| 128 | 64 | 32 | 16 | 8 | 4 | 2 | 1 |`

#	Denary	Binary (8-bit)	Hex
1	217		
2		`10101101`	
3			`5A`
4	202		
5		`00101111`	
6	190		
7			`2D`
8		`10110011`	

Working space:

Activity 2 — Two's complement (8-bit signed, range −128 to +127)

Reminder: **to find −x, invert ALL bits then ADD 1.**

(a) Represent each of these denary values in 8-bit two's complement:

Denary	8-bit two's complement
−20	
−100	
−1	

Working:

(b) Carry out these subtractions using **A + (−B)** in 8-bit two's complement. Show A, the two's complement of B, the addition, and **discard the final carry** out of the MSB. State the denary result.

(i) 35 − 18

```
 A   = ________
-B   = ________   (two's complement of 18)
----------------
Sum  = ________   (carry out: ____ → discard)
```

Denary result = __

(ii) 20 − 50

```
 A   = ________
-B   = ________   (two's complement of 50)
----------------
Sum  = ________
```

Denary result = __ (Is the result negative? How does the MSB tell you?) ___

Activity 3 — Floating-point normalisation

A floating-point number is stored as **value = mantissa × 2^exponent**, with the mantissa and exponent both in **two's complement**, and the binary point assumed **after the first (sign) bit of the mantissa**.

Normalisation rule: a positive number's mantissa must start `0.1…`; a negative number's must start `1.0…`.

You are given the **un-normalised** positive mantissa `0.0001101` (binary point after the first bit). Normalise it.

(a) How many places, and in which direction, must the point move so the mantissa starts `0.1…`? ____

(b) Moving the point that direction — does the exponent **increase** or **decrease**? _____

(c) Write the normalised mantissa: ___

(d) Write the exponent (in denary) needed so the value is unchanged: _____

(e) In one sentence, why do we normalise? _____

Activity 4 — Binary shifts

For each, give the **resulting 8-bit pattern** AND state the **operation/effect** (direction and × or ÷ 2^n). Bits shifted off the end are lost.

(a) `00010110` shifted **logical left by 2**: Result = _ _ Effect = _____ _____

(b) `10110000` shifted **logical right by 3**: Result = _ _ Effect = _____ _____

Activity 5 — Data structures

(a) Match each structure (1–10) to its description (A–J). Write the letter.

Structure	Letter
1. Array	
2. Record	
3. List	
4. Tuple	
5. Linked list	
6. Stack	
7. Queue	
8. Tree	
9. BST	
10. Hash table	

Descriptions: - **A.** FIFO; enqueue at the rear, dequeue at the front. - **B.** Fixed-size, same-type collection accessed by index (0-based), O(1) access. - **C.** Ordered, **immutable** sequence — cannot be changed once created. - **D.** Connected, acyclic structure with a root, parents, children and leaves. - **E.** Keys mapped to indices via a hash function for ~O(1) lookup; two keys to one index = a collision. - **F.** Nodes of data + a pointer to the next; head pointer, last points to null; no random access. - **G.** Collection of named fields that may hold **different** types. - **H.** LIFO; push/pop/peek at the top via a stack pointer. - **I.** Binary tree where the left subtree < node < right subtree. - **J.** Ordered, **mutable**, resizable collection; append/insert/remove/index.

(b) Trace table. Start with an **empty stack** and an **empty queue**. Apply the operations in order to BOTH structures (push/enqueue the same value, pop/dequeue when shown). Record the contents after each step and the value removed.

Operations in order: **add 7, add 3, add 9, remove, add 5, remove.**

Step	Operation	Stack contents (top → bottom)	Stack removed	Queue contents (front → rear)	Queue removed
1	add 7		–		–
2	add 3		–		–
3	add 9		–		–
4	remove				
5	add 5		–		–
6	remove				

Activity 6 — Boolean algebra & logic

(a) Complete the truth tables.

XOR (Q = A ⊕ B)

A	B	Q
0	0	
0	1	
1	0	
1	1	

NAND (Q = ¬(A · B))

A	B	Q
0	0	
0	1	
1	0	
1	1	

(b) Simplify using De Morgan's ("break the bar, change the sign — negate **each** variable and swap AND↔OR"). Show each step.

(i) `¬(A · B) + B`

(ii) `¬(¬A + ¬B)`

(c) Half adder. Complete the outputs for the two inputs A and B.

A	B	Sum (A ⊕ B)	Carry (A · B)
0	0		
0	1		
1	0		
1	1		

State the Boolean expressions: **Sum =** __ **Carry =** ____

Activity 7 — Floating-point addition & normalisation drill

> **Examiner note:** floating-point arithmetic is a documented weak spot — candidates forget to **align exponents** before adding mantissas, and forget to **re-normalise** (and adjust the exponent) afterwards. Work through every step.

All numbers use an **8-bit mantissa** and a **4-bit exponent**, both in **two's complement**, with the binary point after the first (sign) bit of the mantissa.

(a) Add these two floating-point numbers. Follow the steps.

```
Number P:  mantissa 0.1010000   exponent 0011
Number Q:  mantissa 0.1000000   exponent 0001
```

Step 1 — Convert each to denary as a check: P = _ *Q* = _

Step 2 — Align: rewrite **Q** so its exponent matches P's (shift the mantissa right, one place per step):

Q becomes: mantissa __ exponent __

Step 3 — Add the mantissas:

```
  0.1010000
+ _________
-----------
  _________
```

Step 4 — Result (normalise if needed): mantissa __ exponent __

Step 5 — Denary check (should equal P + Q): _____

(b) A calculation elsewhere produces the **un-normalised** result:

```
mantissa 0.0001100   exponent 0101
```

Normalise it (mantissa and exponent), and give the denary value as a check.

Normalised mantissa = __ Exponent = _ *Denary value* = _

(c) Normalise this **negative** number (remember: a normalised negative mantissa starts `1.0…`):

```
mantissa 1.1101000   exponent 0100
```

Normalised mantissa = __ Exponent = _ *Denary value* = _

Activity 8 — Karnaugh map simplification

> **Examiner note:** common errors examiners report — grouping diagonally (not allowed), missing the **wrap-around** groups at the edges/corners, using group sizes that are not powers of 2, and not making groups as **large** as possible. Groups may overlap.

(a) The Boolean function is **Q = ¬A·B·C + A·B·C + A·B·¬C**.

Fill in the 3-variable Karnaugh map (put a 1 in each cell where Q = 1, 0 elsewhere), draw your groups, then write the simplified expression.

	BC=00	**BC=01**	**BC=11**	**BC=10**
A=0				
A=1				

Simplified Q = _____

(b) The completed 4-variable Karnaugh map for output **Q** is shown below (rows AB, columns CD, Gray-code order).

AB \ CD	**00**	**01**	**11**	**10**
00	1	0	0	1
01	0	1	1	0
11	0	0	0	0
10	1	0	0	1

Identify the groups (hint: one group **wraps around all four corners**) and write the simplified expression.

Groups found: _____

Simplified Q = _____

Challenge box

> A programmer claims that the 8-bit two's complement pattern `10000000` is "negative zero, so it equals 0."
>
> **(a)** What denary value does `10000000` actually represent in 8-bit two's complement? Show your reasoning using the −128 place value.
>
> **(b)** Explain why two's complement (unlike sign-and-magnitude) has **only one** representation of zero.
>
> **(c)** A colour is written `#1E90FF`. Convert the green channel `90` (hex) to denary, and the blue channel `FF` to denary. Why is hex used for colours rather than raw binary?

Section B — Exam practice (30 marks)

Answer all questions in the spaces provided. In questions marked with an asterisk (*) the quality of your extended response will be assessed.

Q1 Describe **one** difference between the ASCII and Unicode character sets. **[2]** (AO1)

_____

_____

_____

_____

_____

Q2 A weather station records air temperatures between −60 °C and +60 °C and stores each reading as an **8-bit two's complement** integer.

Show how the temperature **−45** is stored. *You must show your working.* **[3]** (AO2)

_____

_____

_____

_____

Working space:

Q3 A games engine stores numbers in a floating-point format with an **8-bit mantissa** and a **4-bit exponent**, both in two's complement, with the binary point after the first (sign) bit of the mantissa.

Calculate the denary value of each of the following stored numbers. *You must show your working.*

(i) mantissa `01011000`, exponent `0011` **[2]** (AO2)

Working space:

(ii) mantissa `10110000`, exponent `0011` **[2]** (AO2)

Working space:

Q4 Use Boolean algebra to simplify the following expression, showing each step and naming the identity or rule used.

X = A·B + A·¬B + ¬A·B [3] (AO2)

_____

_____

_____

Q5 A text editor provides an *undo* feature. Every time the user performs an action (e.g. typing a word, deleting a line) the action is recorded; pressing *undo* reverses the most recent action first.

Explain why a **stack** is an appropriate data structure for this feature, describing the operations used when an action is performed and when *undo* is pressed. **[3]** (AO2)

Q6 State the purpose of a **D-type flip-flop**, and describe the role of the **clock** input. **[2]** (AO1)

Q7 An online game stores each player's high score. Player names (the keys) are stored in a **hash table** so that a player's score can be found quickly.

Explain how the hash table is used to **store** and **retrieve** a player's score, and what happens when a **collision** occurs. **[4]** (AO2)

Q8* A developer is building a contacts app for a phone. The app must:

- find a contact's details from their name **as quickly as possible**, and
- display the full contact list **in alphabetical order** whenever it is opened.

The developer is deciding whether to store the contacts in a **hash table** or a **binary search tree (BST)**.

Discuss the suitability of each data structure for this app and recommend, with justification, which the developer should use. **[9]** (AO3)

Section B total: 30 marks
